

Development and Enhancement of a Lightweight Robotic Arm with Torque Control Exploration

Tim Heger

Chair of Automatic Control, Technical University of Munich
3. November 2025

Abstract

This thesis continues the development of a lightweight robotic arm, building on the master thesis of Samuel Zeitler. The existing prototype is enhanced with a focus on improving robustness and durability while maintaining as much of the lightweight structure as possible. The mechanical design is revised to include ball bearings and aluminum axes among other improvements, and a planetary gearbox is added to the yaw joint to increase the available torque by a factor of 8. Furthermore, a modular end-effector design is introduced, featuring a two-fingered gripper. The MATLAB-based software suite is extended with more advanced control algorithms, utilizing the indirect torque measurement of the servo motors. This improved design is assembled to create a reliable and modular experimental platform for the exploration of robotic arm control strategies at the Chair of Automatic Control Theory.

Contents		Torque-Based Control	10
		Joint Space PD-Control	11
Introduction	1	Cartesian Space PD-Control . . .	11
		TCP Force Control	12
Hardware Enhancements	2		
Shoulder Joint	2	Conclusion	12
Yaw Joint	3	Contributions	13
Elbow Joint	4	Limitations	13
End-Effector	5	Future Work	13
Construction	5		
Torque Measurement	6	Introduction	
Experiment setup	6	The foundation for the work presented	
First Measurements	6	in this thesis is the lightweight robotic	
Validating Assumptions	7	arm developed throughout several student	
Determining Conversion Factors	9	projects, the latest being the semester the-	
Implementation	10	sis[6] and master thesis[7] of Samuel Zeitler.	
		Over the course of these previous projects,	

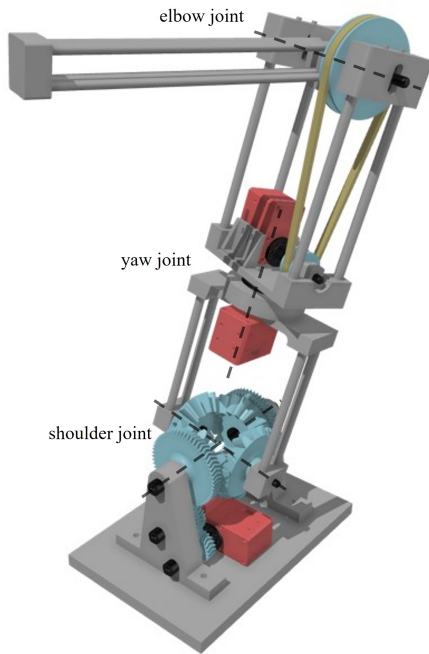


Figure 1: Lightweight robotic arm prototype as designed by Samuel Zeitler

a lightweight robotic arm was developed as seen in Figure 1.

The robotic arm consists of several joints, resulting in four degrees of freedom in total. The shoulder joint has two degrees of freedom, the yaw joint and elbow joint each have one. This ensures a range of motion wide enough for the demonstration purposes. This existing prototype was designed with the goal of mounting it on a drone, which is why the focus was on a lightweight design. Since the robot arm is now intended to be used as an experimental platform for the exploration of robotic arm control strategies at the Chair of Automatic Control Theory, the main focus of this thesis is to enhance the existing prototype to improve accuracy, robustness and durability.

The existing prototype is controlled using a MATLAB-based software suite, which was also developed in previous student projects. This software utilizes the servos' integrated velocity control for a velocity-based control approach of the robotic arm. In this thesis, the current control capabilities of the servos

are explored to enable torque-based control strategies. Specifically, the indirect torque measurement capabilities of the servos are investigated and utilized to implement basic torque reading functionality.

The following chapters will detail the enhancements made to the hardware of the robotic arm, the implementation of a basic torque reading strategy and the exploration of torque-based control.

Hardware Enhancements

The main focus of the enhancements is on the axes and bearings, which were previously 3D-printed and are now replaced with more robust components. The Yaw joint was also significantly modified, now featuring a planetary gearbox to increase the torque available at the joint.

Shoulder Joint

The shoulder joint is the most complex part of the robotic arm, featuring two degrees of freedom, tilting the arm in two axes. Most of the parts of this Joint are stationary, meaning they are not moving with the arm. As a first enhancement, all stationary parts (the side braces and the base plate) were replaced with aluminum components, which are more robust and durable than the previous 3D-printed parts. The two joint axes of the shoulder joint are driven by servo motors, which drive several gears to achieve the desired range of motion. There are six gear axes used to fix the gears to the side braces. all of these axes were previously 3D-printed and are now replaced with aluminum components. Furthermore, all axes of the shoulder joint are now supported by ball bearings, as seen in Figure 2.

The centerpiece of the joint are the four bevel gears, which are the key to allowing the two degrees of freedom. The two fixed gears are mounted to the side braces as mentioned above, while the two mov-

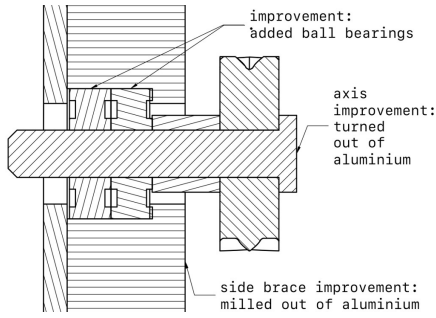


Figure 2: Exemplary view of the improvements made to the shoulder joint, showing the section view of one gear axis.

ing gears mount the arm itself. Previously, these gears both had their own axis, fixed to the two sides of the arm respectively. Now, they share a common axis, where one of them is fixed and the other is free to rotate using ball bearings, as seen in Figure 3. This change improves the alignment, fixing the gears in place in axial direction, negating the need for tensioning tape as used in the previous assembly[7].

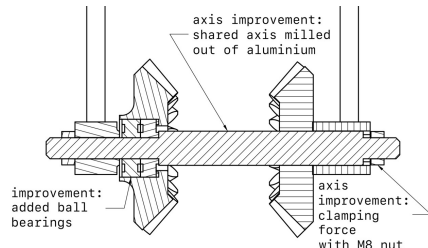


Figure 3: Section view of the improvements made to the bevel axis.

Yaw Joint

The yaw joint is the second joint of the robotic arm, featuring one degree of freedom allowing the arm to rotate around its vertical axis. Previously, the yaw joint had a 1:1 gear ratio, which was changed to a 8:1 gear ratio to increase the torque available at the joint using a planetary gearbox shown in Figure 4. This gearbox is made up of a sun gear driven by the motor, three planet gears which drive the rest of the arm, and a ring gear fixed to the lower part of

the arm. These gears were designed using a gearbox template for OpenSCAD[5] And 3D-printed at the workshop of the institute. The planetary gears themselves are mounted on the carrier using ball bearings marked in blue in the figure, the motor axis used to drive the sun gear is marked in black. This axis is also turned out of aluminum to increase durability. The carrier and the gears are 3D-printed. The axes of the planet gears are made up of screws, which directly connect the carrier to the top part of the arm.

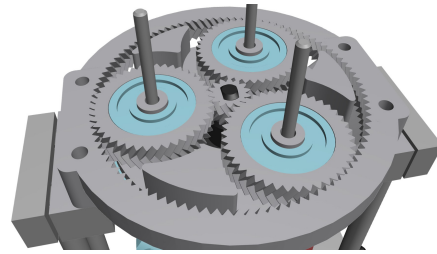


Figure 4: View of the gears in the planetary gearbox.

Furthermore, the axis of the yaw joint is now supported by axial bearings, which improves the precision of the robot. Two axial bearings are used, one above and one below the planetary gear carrier. The bearings are marked in blue in Figure 5.

The four axes marked in Figure 5 show screws, two of which connect the outer gear ring to the base plate of the arm, while the other two connect the lid of the planetary gearbox to the base plate. For future development, the lid of the planetary gearbox could be fixed using four screws instead of two, to improve the stability of the construction.

To further improve the 3D-printed planetary gearbox, silicone spray was applied to the gears to reduce friction and wear. The impact on the friction was measured using the current sensor of the servo motor, as the internal friction of the gearbox is directly proportional to the current needed to drive the gearbox. As seen in Figure 6, the application of silicone spray resulted in a

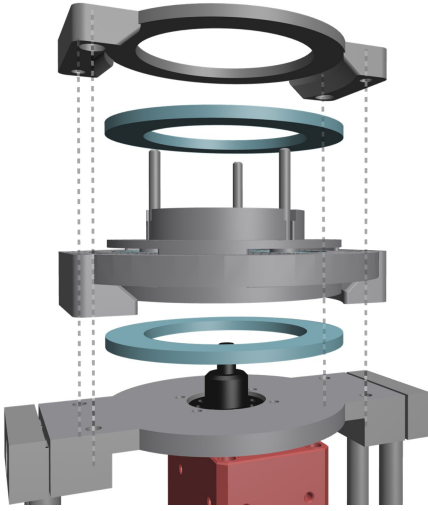


Figure 5: Exploded view of the planetary gearbox.

slight reduction of measured current, therefore proving a reduction of friction.

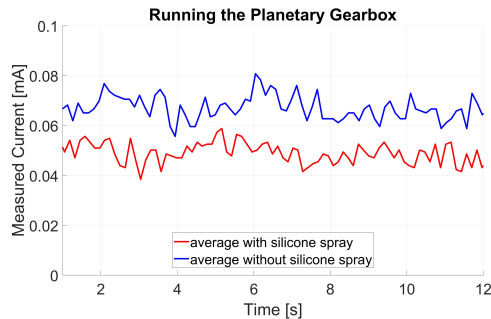


Figure 6: Assessment of the planetary gearbox internal resistance before and after applying silicone spray.

For future developments, one possible improvement in terms of assembly could be to substitute the three screws used as planet gear axes with screws featuring a hexagon head, and adapt the bottom plate of the planetary gear carrier in such a way that the screwheads are locked in position and the screws cannot turn. This would ease the assembly process, as the screws would not need to be held in place while tightening the nuts on the other side.

Elbow Joint

The elbow joint is the third joint of the robotic arm, featuring one degree of freedom allowing the arm to bend. This joint is driven by a belt drive, for which the gears were previously 3D-printed and are now replaced with commercially available aluminum gears. Two axes are part of this joint, one motor axis which drives the belt and one output axis to drive the arm. Both of these are replaced with aluminum axes.

The motor axis is now supported by two ball bearings as seen in Figure 7. The connecting piece which holds the motor axis connects to carbon fiber rods which continue upwards. This connection now introduces a clamping mechanism instead of the previous glue connection for better modularity.

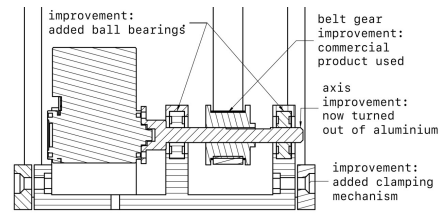


Figure 7: Section view of the construction surrounding the motor axis.

The elbow axis sits at the other end of the belt drive, and is therefore responsible for tensioning the belt. This tensioning mechanism consists of screws and self-securing nuts, supporting the ball bearings and applying the necessary force to keep the belt tensioned, see Figure 8

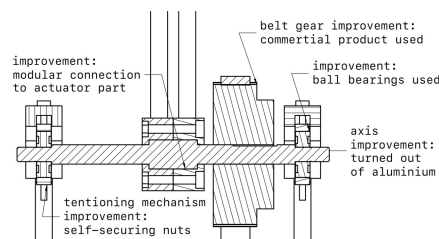


Figure 8: Section view of the construction surrounding the elbow axis.

For future development, the connection to the actuator part (see Figure 8) could be constructed to have a larger overlap between the aluminum axis and the 3D-printed carbon fiber rod holder. This would reduce the endpoint force of the aluminum pressing against the 3D-printed part. Reducing this force will reduce the wear in the 3D-printed part, which is responsible for end-effector inaccuracies. The current solution to this inaccuracy problem is epoxy glue, however this reduces the modularity. Another improvement could be to increase the length of the housing around the tensioning mechanism. Currently, the self-securing nuts do not overlap with the housing. This leads to the possibility of these nuts to turn in place, as the current fastening method is not secure enough, glue is needed in the current design to hold these nuts in place. If the housing were to overlap with the screws, it would prevent them from turning freely, fixing this problem without the need for glue.

End-Effector

The end-effector (TCP) is the last part of the robotic arm, which is used to interact with the environment. To easily exchange the end-effector, the clamping mechanism introduced in Figure 7 is used here as well to connect the end-effector to the carbon fiber rods. The current design features a the two-fingered gripper manufactured by makeblock[3] as seen in Figure 9.

Since the PCB board used in this project only supports dynamixel servos and no external motors, the gripper is controlled manually using a switch at the base of the robot arm.

For future development, a different microcontroller could be used to incorporate gripper control into the software.

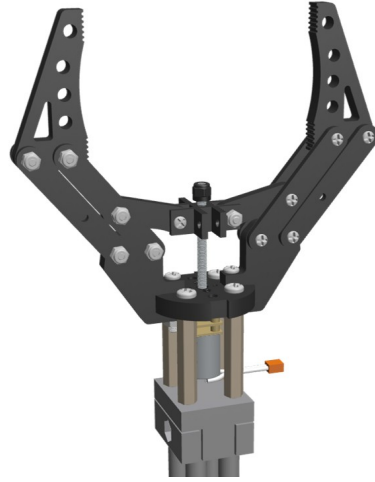


Figure 9: Robot end-effector featuring a simple gripper.

Construction

With all hardware enhancements made, the improved version of the robotic arm can be seen in Figure 10.

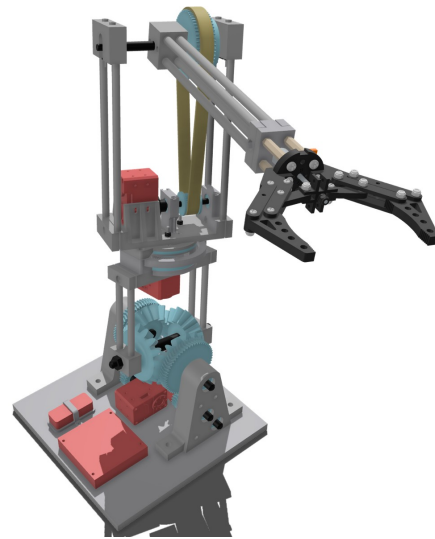


Figure 10: Final version of the robotic arm with all hardware improvements modelled.

This version has been built in collaboration with the workshop of the institute, with the help of their machinery and expertise. The built robotic arm can be seen in Figure 11.

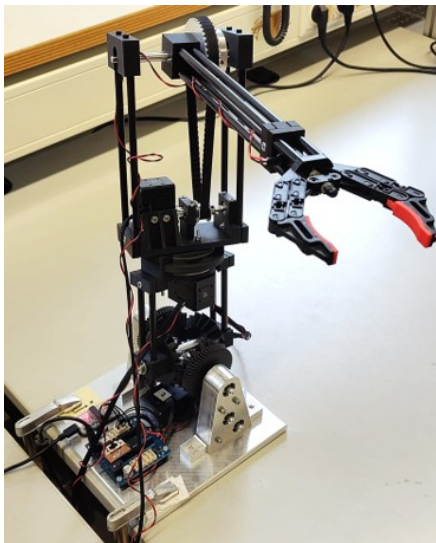


Figure 11: Built version of the robotic arm with all hardware improvements implemented.

Torque Measurement

The previous version of the robot used a velocity-based control approach, setting a target velocity for the controller. The new version of the robot features servo motors capable of current measurement and current control, which can be used to indirectly measure the torque applied to the motors. Several experiments were conducted to understand the relationship between the measured current and the actual torque, detailed in the following sections.

Experiment setup

The experiment setup consists of a servo motor with a wheel attached as seen in Figure 12. The motor is securely mounted on the base plate, which in turn is clamped to a table. A second table of the same height is placed opposite the wheel and on it rests the foot which holds up the free end of the axis.

The wheel pulls a set weight by a string, therefore applying a constant torque to the motor. Depending on the direction in which the string is wrapped around the

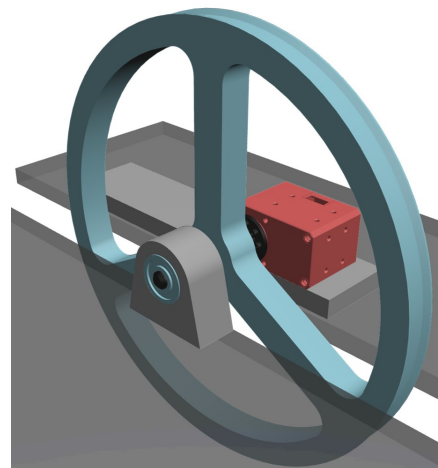


Figure 12: Experiment setup for measuring torque.

wheel, the applied torque can be either clockwise or anticlockwise. The current is then measured while the weight is being moved up and down, both for clockwise and for anticlockwise torque.

First Measurements

For the first experiments, a metal weight of 1.056 kg was used, which applies a torque of 1.036 Nm to the motor. The motor is controlled using the internal velocity control, which is the same control method used in the previous software version of the robot. As a first experiment, the motor was simply moved upwards and downwards with the weight attached, holding the weight at the topmost position for 10 seconds. The experiment has been repeated 4 times and the results can be seen in Figure 13.

As can be seen in the figure, there is a significant difference between the current measurements for hoisting the weight upwards and lowering it downwards. Each measurement has a slightly different starting time for the upward movement, which is due to the fact that the weight starts on the ground and the string is initially slack, therefore the motor needs to wind up some string before the weight starts moving. It can also be noted that during move-

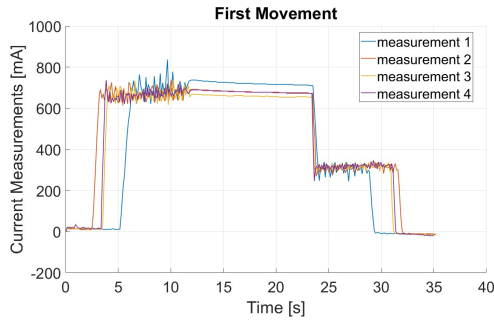


Figure 13: Current measurements for moving an approx. 1 kg weight upwards and downwards.

ment (e.g. from 5s to 10s in Figure 13), the current measurements are a bit noisy, while when the weight is held in place (e.g. from 12s to 22s in Figure 13), the current measurements are much more stable. One possible explanation for the difference between the upwards and downwards movement is to assume that there is some internal friction in the motor, which causes a higher current draw when the motor is working against the external torque. However it is interesting to note that the current draw when holding the weight in place is equal to the current draw when hoisting the weight upwards, at least in the experiments so far. To further investigate this, the weight is lifted by hand, temporarily reducing the applied torque, while the motor is holding the weight in place. The results of this experiment can be seen in Figure 14.

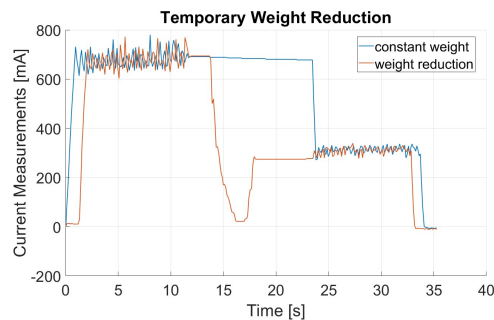


Figure 14: Current measurements when temporarily reducing the applied torque by lifting the weight by hand.

By the 20 second mark, the full weight is already being held in place by the motor again, but the current draw has not increased to the previous value. This experiment shows, that the current measurements when holding the weight in place are not consistent, but depend on the previous movement. When the weight is lifted by hand, the current draw decreases, as expected, but when the weight is let go again, the current draw increases again to a lower value than before, equal to the current draw when lowering the weight. This means that it is not possible to determine one single current-torque relation for static measurements. It also strengthens the assumption that there is some internal friction in the motor, since the current draw when holding the weight in place after lowering it is lower than when holding it in place after hoisting it upwards.

Validating Assumptions

There are a couple of assumptions that need to be validated before proceeding with further experiments.

- First, it needs to be ensured that all four motors behave similarly, so that the results of experiments with one motor can be generalized to all four motors.
- Second, it needs to be validated that the measured current is only dependent on the applied torque, and not on other forces, mainly radial loads on the motor axis.
- Third, it needs to be validated that the motor behaves similarly for all control modes: position control, current control and velocity control.

To validate the first assumption, all four motors were tested one after another with the same movement profile and weight. The movement profile was slightly modified from the previous experiments to include

static phases after both upwards and downwards movement: The weight is first lifted, then held in place, then lowered halfway, held again, and finally lowered all the way down. The results of this experiment can be seen in Figure 15.

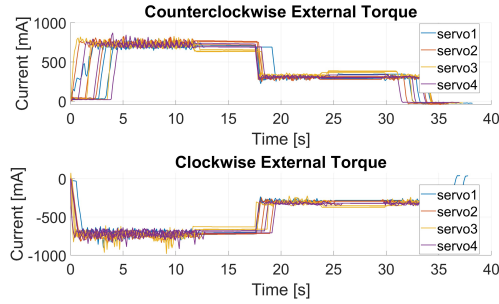


Figure 15: Current measurements for all four motors with a 1 kg weight.

As can be seen in the figure, all four motors behave very similarly, validating the assumption that they are interchangeable. Thus only one motor was used for the following experiments. Furthermore, the behavior for both directions of applied torque (clockwise and anticlockwise) is also very similar, validating the assumption that the motor behaves similarly for both directions of applied torque. Therefore, only one direction of applied torque was used for validating the remaining assumptions.

To validate the second assumption, the string was tied around the axis such that a radial load is applied to the motor axis, instead of a torque. The motor was then moved in both directions, with and without the radial load. The results of this experiment can be seen in Figure 16.

As can be seen in the figure, there is no significant difference between the current measurements with and without radial load, validating the assumption that the measured current is only dependent on the applied torque. It can also be seen that the measurements are very low (max 40mA) compared to the previous experiments (up to 700mA), which is expected

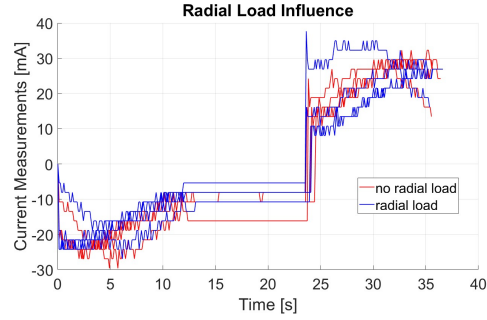


Figure 16: Current measurements with and without radial load on the motor axis.

since the motor is not working against any external torque.

To validate the third assumption, two more experiments were conducted, one with position control and one with current control. The current control experiment was conducted using the same movement profile as before, and can therefore be easily compared to the previous velocity control experiments. As can be seen in Figure 17, the results are very similar to the previous velocity control experiments, validating the assumption that the motor behaves similarly for both control modes.

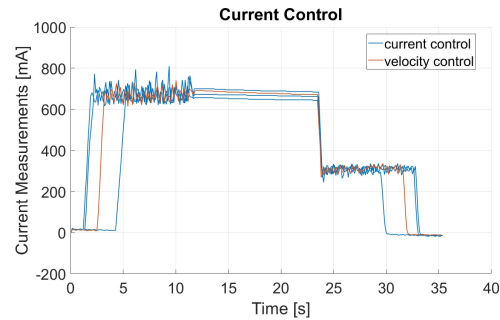


Figure 17: Current Control

The position control experiment was conducted by moving the motor to a set position and holding it there for a short moment. This experiment was also used to further check the behavior of the measured current when holding a changing weight in place. Similar to Figure 14, the weight was lifted by hand while the motor was holding

it in place. The results of this experiment can be seen in Figure 18.

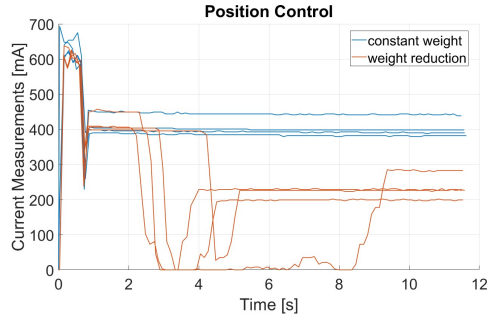


Figure 18: Position Control

As can be seen in the figure, the results while the motor is moving are very similar to the previous experiments, validating the assumption that the motor behaves similarly for position control as well, at least during movement. The results while holding the weight in place without changing it land somewhere in the 400mA region, which is different than the previous experiments. Without changing the weight manually, the current for holding the weight in place was either around 700mA (after hoisting upwards) or around 300mA (after lowering downwards). Similarly to the experiment shown in Figure 14, the current necessary for holding the 1 kg weight decreases once it has been lifted by hand. This leads to the conclusion, that while the motor is not moving, the applied torque can not be consistently determined from the measured current, at least not with the information available from the servo alone.

For future works, the behavior of measurements while the motor is not moving could be further investigated. One possible approach could be to use a model of the gear resistance and previous movements to better approximate the conversion factor necessary to determine the applied torque from the measured current. Another approach could be to use an external torque sensor to directly measure the applied torque, which would evade the problem of gearbox friction.

Determining Conversion Factors

This work will focus on the behavior of the motor while it is moving, where the applied torque can be consistently determined from the measured current. The goal is to determine the conversion factors from current to torque for both cases: working against applied torque and working with applied torque.

To determine these conversion factors, a new movement profile is used. The weight is only moved dynamically, without any static holding phases. The weight is first hoisted upwards for the entire distance, then lowered downwards for half the distance, hoisted upwards again for half the distance, and finally lowered downwards for the entire distance back to the starting position.

For the first experiment, this new movement profile was executed for both directions of applied torque (clockwise and anticlockwise). The absolute results are gathered and averaged for the two cases (hoisting and lowering weight), as can be seen in Figure 19.

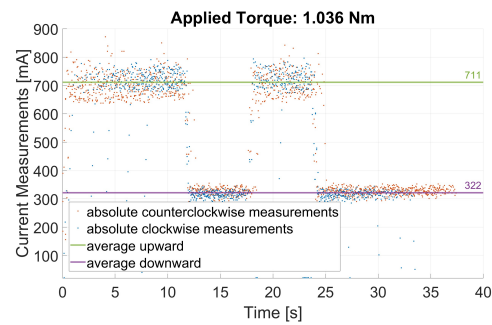


Figure 19: Current measurements determining the average current draw for working against applied torque (moving upwards) and working with applied torque (moving downwards).

From this experiment, the average current draw for both cases could be determined and thus the conversion factor from current to torque can be computed. However, since the experiment was only conducted

with a 1 kg weight, it cannot be guaranteed that the relation is linear. Therefore, a second experiment was conducted, where the same movement profile was executed with different weights ranging from 100g to 1 kg. The measurements taken from this experiment are plotted torque vs current draw, as shown in Figure 20.

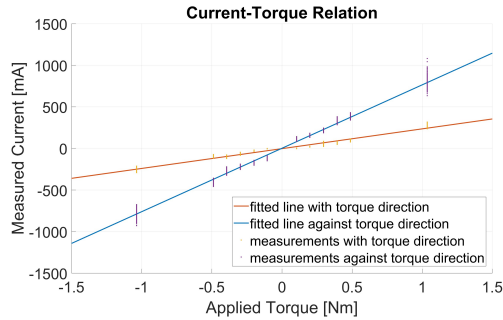


Figure 20: Current measurements for different weights, determining the relation between applied torque and current draw.

The results show a clear difference between the two cases of working against applied torque (hoisting upwards) and working with applied torque (lowering downwards). This can be explained by the internal friction of the motor and gearbox: when working against the applied torque, the motor needs to overcome both the external torque and the internal friction, resulting in a higher current draw for the same applied torque. Similarly, when working with the applied torque, the internal friction reduces the necessary current draw. This behavior is taken into account by determining two separate conversion factors. For the two cases it is clear that the relation between applied torque and current draw is linear, therefore they can be modeled with two linear factors:

- Working against applied torque (hoisting upwards): $m_{resisting} = 762.8 \frac{A}{Nm}$
- Working with applied torque (lowering downwards): $m_{assisting} = 238.1 \frac{A}{Nm}$

Since all static torque measurements measured a current value somewhere between

these two factors, it is safe to assume that the conversion factor for the static case, while not being constant, lies somewhere between these two factors.

- Static case (holding weight in place): $m_{resting} \in [238.1, 762.8] \frac{A}{Nm}$

Implementation

A set of functions was implemented to read the torque applied to a motor, using the conversion factors determined previously. In the modular sense of the existing code[7], functions were added to read the current and calculate the torque of each servo. Using the signs of the motors velocity and current measurements to determine the direction of the applied torque, it is decided which conversion factor to use. For the static case (motor velocity = 0), the function uses the upper limit resting factor, since this is the more conservative choice.

Another function was added which uses the gear ratios of each joint to convert the torque applied to the motor axis into the torque applied to the joint axis.

Finally, a program was written which uses the Jacobian to compute the cartesian force at the end-effector based on the joint torques using the following formula[2]:

$$F = (J^T)^+ \tau \quad (1)$$

Where τ is the vector of joint torques, $(J^T)^+$ is the pseudoinverse of the transposed Jacobian matrix and F is the vector of cartesian forces at the end-effector.

Torque-Based Control

As a starting point for torque control, several approaches for a PD-controller were implemented. These all relied on the current control mode of the servos, therefore some basic functions were implemented in

the modular code structure to set the control mode of all servos to current control, as well as to set a target torque for all joints.

Joint Space PD-Control

The first approach was a simple PD-controller in joint space, which computes a target torque for each joint based on the current joint position and velocity, as well as a desired joint position. The desired torque for each joint is then set as calculated using the following formula [4]:

$$\tau_{target} = -K_p \Delta q - K_d \dot{q}_{current} \quad (2)$$

Where τ_{target} is the vector of target torques for each joint, K_p and K_d are the proportional and derivative gain vectors, Δq is the vector of joint position errors and $\dot{q}_{current}$ is the vector of current joint velocities. The joint position errors are calculated as the difference between the positions at the beginning of the program and the current joint positions, normalized over the joint limits. The normalization is done to ensure balanced control across the joints with different motion ranges. The current joint positions and velocities are calculated from the servo readings. Finally, the gains K_p and K_d are set as constants in the program, which were manually tuned to achieve a stable behavior.

This program makes the robot hold its initial position, while being compliant in each joint. If any joint of the robot is moved away from its initial position, this joint will snap back like a damped spring.

Cartesian Space PD-Control

The second approach was a PD-controller in cartesian space, which computes a target force for the end-effector based on the current end-effector position and velocity, as well as a desired end-effector position. The following formula is used to calculate the target force[4]:

$$F_{target} = -K_p \Delta x - K_d \dot{x}_{current} \quad (3)$$

Where F_{target} is the vector of target forces for the end-effector, K_p and K_d are the proportional and derivative gain vectors, Δx is the vector of TCP position errors and $\dot{x}_{current}$ is the vector of current TCP velocities. The position errors are calculated as the difference between the position at the beginning of the program and the current position. The current position is calculated using forward kinematics[7], while the current velocity is calculated using the backward difference derivative[1]:

$$\dot{x}_{current} = \frac{x_{current} - x_{previous}}{dt} \quad (4)$$

Where $x_{current}$ is the current TCP position, $x_{previous}$ is the TCP position from the previous iteration and dt is the time difference between the two iterations. The gains K_p and K_d are again set as constants in the program, which were manually tuned to achieve a stable behavior.

The target force is then converted to a target torque for each joint using a nullspace optimizer. This approach was used to utilize the redundancy of the robot to keep the arm as upright as possible. The nullspace optimizer calculates the target joint torques using the following formula[7]:

$$\tau_{target} = \tau_{task} + \tau_{null} \quad (5)$$

$$\tau_{task} = J^T F_{target} \quad (6)$$

$$\tau_{null} = k_{null} N \frac{\partial H}{\partial q} \quad (7)$$

Where τ_{target} is the vector of target torques for each joint, F_{target} is the vector of target forces for the end-effector, and J is the Jacobian matrix. k_{null} is a constant gain for the nullspace control, N is the nullspace projection matrix, H is a cost function to keep the joints close to a desired position and q is the vector of current joint positions. The nullspace projection matrix is calculated using the following formula:

$$N = I - J^T (J J^T + \lambda_{damp}^2 I)^{-1} J \quad (8)$$

Where λ_{damp} is a damping factor to improve numerical stability and I is the iden-

tity matrix. The cost function and its derivative are defined as:

$$H = 0.5 (q - q_{des})^T W (q - q_{des}) \quad (9)$$

$$\frac{\partial H}{\partial q} = W (q - q_{des}) \quad (10)$$

Where $q_{des} = (0, 0, 0, 0)^T$ is the desired upright position and q is the vector of current joint positions. The weight matrix

$$W = \text{diag} \left(\frac{1}{q_{max}^2} \right) \quad (11)$$

is used to normalize the cost function over the joint limits q_{max} .

This program makes the robot hold its initial end-effector position, while being compliant in cartesian space. If the end-effector is moved away from its initial position, the robot will snap back like a spring. however if the arm is moved along its degree of freedom while holding the end-effector position, it is quite easy to move. The arm will then only slowly return to the most upright position, due to the nullspace control.

TCP Force Control

The third approach was a force controller for the end-effector, which is similar to the cartesian space PD-controller, but instead of holding a position, it holds a constant force at the TCP. For this program to behave in a stable manner, a Cartesian space PD-controller is used, with the desired end-effector position not being set as a point in space, but rather a line defined by the starting point and the vector of the desired force as a direction vector. The target force is then calculated using the same formula as before, but the position error is calculated as the distance from the current TCP position to the closest point on the line:

$$\Delta x_{line} = \Delta x - \frac{(\Delta x \cdot F_{des})}{\text{norm}(F_{des})^2} \cdot F_{des} \quad (12)$$

Similarly the end-effector velocity is calculated as the change in distance to the line over time:

$$\dot{x}_{line} = \frac{\Delta x_{line,curr} - \Delta x_{line,prev}}{dt} \quad (13)$$

Where $\Delta x_{line,curr}$ is the current distance to the line and $\Delta x_{line,prev}$ is the distance to the line from the previous iteration. The target force is then calculated using the same formula as before, but using the distance to the line and its derivative instead of the position and velocity and adding the desired force:

$$F_{target} = F_{pd} + F_{des} \quad (14)$$

$$F_{pd} = -K_p \Delta x_{line} - K_d \dot{x}_{line} \quad (15)$$

Where F_{des} is the desired constant force at the TCP. The target force is then converted to a target torque for each joint using the same nullspace optimizer as before.

The program makes the robot hold a constant force at the endeffector, while being compliant in cartesian space. The robot will move from the starting position in the direction of the desired force, until it meets a resisting force equal to the desired force, or the joint limits (or a singularity) are reached. If the TCP is moved away from the line of the desired force, the robot will resist the movement and try to return to the line. It can still be moved easily along the degree of freedom of the arm, as long as the end-effector is kept on the line of the desired force. It will only slowly return to the most upright position, due to the nullspace control.

Conclusion

This paper presented a comprehensive study on the design and implementation of a robotic arm for lab testing and education. It discussed the hardware components, control algorithms, and the integration of sensors for feedback. The results demonstrate the effectiveness of the proposed system in performing complex tasks with high precision and reliability.

Contributions

In summary, the hardware improvements made to the robotic arm include:

- Replacement of 3D-printed axes with aluminum axes for increased durability.
- Introduction of ball bearings to support the axes and reduce friction.
- Implementation of a clamping mechanism for better modularity and easier assembly.
- Modification of the yaw joint to include a 8:1 planetary gearbox to increase available torque.
- Enhancement of the tensioning mechanism for the elbow joint to improve reliability.
- Integration of a gripper with manual control, with potential for future software control.

The software contributions include:

- Development of a torque measurement method using the current sensing capabilities of the servo motors and the results from extensive experiments.
- Extension of the existing robot control framework to support current control mode and torque commands.
- Implementation of various torque-based control strategies, namely joint space PD-control, Cartesian space PD-control, and end-effector force control.

Limitations

Despite the significant improvements made, there are still some limitations to the current design. Mainly, the stiffness of the robotic arm is still limited by the use of 3D-printed components in some

areas. Therefore, the maximum load the arm can handle is still relatively low if the precision of the end-effector position is to be maintained.

Additionally, the torque measurement method relies on the measured current of the servo motors, which leads to inaccuracies due to unmodeled gearbox friction and other factors. To be precise, the torque measurement is ambiguous in the static case, as the same current can correspond to different torques depending on the previous movement and friction. This limitation could affect the performance of future torque-based control strategies relying on this measurement.

Future Work

Future work could focus on further improving the hardware design to increase the stiffness and load capacity of the robotic arm. Concrete ideas for further hardware improvements can be found in the Chapter Hardware Enhancements.

On the software side, future work could focus on refining the torque measurement method to improve accuracy. This could involve developing a more sophisticated model of the motor and gearbox dynamics or including additional sensors for direct torque measurement.

Furthermore, advanced control strategies using torque sensor data could be explored, such as model-based control or learning-based approaches, to enhance the performance and adaptability of the robotic arm in various tasks. Specifically, implementing advanced impedance control or admittance control could be beneficial for tasks involving interaction with the environment.

Lastly, advanced path planning and motion planning algorithms could be integrated into the control framework to enable more complex and dynamic tasks for the robotic arm.

References

- [1] Lohmann, Prof. Dr.-Ing. habil. B. *Skript zum Praktikum Reglerimplementierung auf Microcontrollern*. Technische Universität München - Lehrstuhl für Regelungstechnik, Sept. 5, 2023.
- [2] Lynch, Kevin M. and Park, Frank C. *Modern Robotics: Mechanics, Planning, and Control*. 1st ed. Cambridge University Press, May 25, 2017. ISBN: 978-1-316-66123-9 978-1-107-15630-2 978-1-316-60984-2. DOI: 10 . 1017 / 9781316661239.
- [3] Makeblock, MKB-86502. *Datenblatt zu Makeblock Robotergreifer, 600 U/min, Greifweite 67mm, Tragkraft 1,5kg, N20-Motor, 12 V, 110 maA*.
- [4] Murray, Richard M., Li, Zexiang, and Sastry, S. Shankar. *A Mathematical Introduction to Robotic Manipulation*. 1st ed. CRC Press, Dec. 14, 2017. ISBN: 978-1-315-13637-0. DOI: 10 . 1201 / 9781315136370.
- [5] Thingiverse.com. *Customizable Planetary Gearbox by Anteino*. Thingiverse. URL: <https://www.thingiverse.com/thing:1361044> (visited on 10/27/2025).
- [6] Zeitler, Samuel. “Development of a Robotic Arm towards Aerial Manipulation”. Semester Thesis. Technical University Munich.
- [7] Zeitler, Samuel. “Towards Dual-Arm Aerial Manipulation: An Exploration in Robotic Arm Design”. MA thesis. Technical University Munich.